


**CSE439s: Design of Compilers**

**Tiny Language Compiler - Scanner Milestone**

**GitHub Repository:** [https://github.com/Alaa-AHK/-Compilers-_The-Scanner-Project.git](https://github.com/Alaa-AHK/-Compilers-_The-Scanner-Project.git)

## Team members

| Name | Student ID |
|---|---|
| Menna Ayman Hassan Radwan | 2200236 |
| Aaesha Mahmod Bedair | 2200510 |
| Aya Mohamed Mohamed Kamel | 2200716 |
| Yomna Ahmed Farag Ali | 2200224 |
| Alaa Abdelhakeem | 2200498 |
| Rola Ahmed Kassem Osman | 2200658 |

## Introduction

This project implements the scanner, the first phase of a compiler, for the TINY programming language.

A scanner reads raw source code one character at a time and groups those characters into tokens: the smallest meaningful units of the language, such as a keyword, an identifier, a number or an operator. Everything that carries no meaning for later phases, namely whitespace and comments, is discarded. The parser that would follow consumes these tokens instead of raw text.

The program is written in Python using only the standard library, so it packages into a standalone Windows executable that runs without installing any third-party tools. It takes a `.tiny` or `.txt` source file and produces a text file listing every token as a `value, type` pair.

## Objectives and deliverables

The specification asks for two outputs from one input: TINY source code in, a `getToken` function and a token file out.

**outputs**

1. A `getToken` function that returns the next token from the input and computes its attributes, including the token's string value.
2. A file listing every token as a `tokenvalue, tokentype` pair, for example `x , Identifier`, `:=, assign` and `4, number`.

## Token specification

TINY has 19 token types: seven reserved keywords, ten special symbols, and two dynamic kinds whose value comes from the source text. The third column is the exact spelling written to the output file.

| TokenType | Example value | Written to the output file as |
|---|---|---|
| SEMICOLON | `;` | SEMICOLON |
| IF | `if` | IF |
| THEN | `then` | THEN |
| END | `end` | END |
| REPEAT | `repeat` | REPEAT |
| UNTIL | `until` | UNTIL |
| IDENTIFIER | `x`, `abc`, `xyz` | Identifier |
| ASSIGN | `:=` | assign |
| READ | `read` | READ |
| WRITE | `write` | WRITE |

| TokenType | Example value | Written to the output file as |
|---|---|---|
| LESSTHAN | `<` | LESSTHAN |
| EQUAL | `=` | EQUAL |
| PLUS | `+` | PLUS |
| MINUS | `-` | MINUS |
| MULT | `*` | MULT |
| DIV | `/` | DIV |
| OPENBRACKET | `(` | OPENBRACKET |
| CLOSEDBRACKET | `)` | CLOSEDBRACKET |
| NUMBER | `12`, `289` | number |

## getToken()

`getToken()` returns exactly one token per call and leaves the scanner positioned at the start of the next one.

1. Record the current line and column as the token's starting position, and enter the START state.
2. Consume one character at a time, moving between DFA states, until a complete lexeme is recognised.
3. When a lexeme ends at a character that belongs to the *next* token, push that character back so the following call sees it.
4. For a run of letters, look the lexeme up in the keyword table: a hit returns that keyword's type, a miss returns IDENTIFIER.
5. When the DFA recognizes a complete token it enters the DONE state, which ends the scanning loop.
6. Return a `Token` carrying its type, string value, line and column.

The scanner tracks position with three fields. `cursor` is the index of the next character to read, while `line` and `column` are the human-readable coordinates stamped onto each token. Consuming a newline increments `line` and resets `column` to 1.

## Lexical error handling

Any lexeme the DFA cannot classify becomes an ERROR token carrying the offending text and its position. Four conditions produce one.

| Condition | Example input | ERROR token value |
|---|---|---|
| A character outside the language alphabet | `@`, `$`, `%` | the character itself |
| A lone `:` not followed by `=` | `x : 5` | `:` |

| Condition | Example input | ERROR token value |
|---|---|---|
| A `:` as the final character of the input | a file ending in `:` | `:` |
| A `{` with no closing `}` | `{ unterminated` | `Unterminated comment '{...}'` |

Scanning halts at the first error. `scan_all()` breaks out of its loop as soon as `getToken()` hands back an ERROR token, so the token list ends there and the rest of the file is left unscanned.

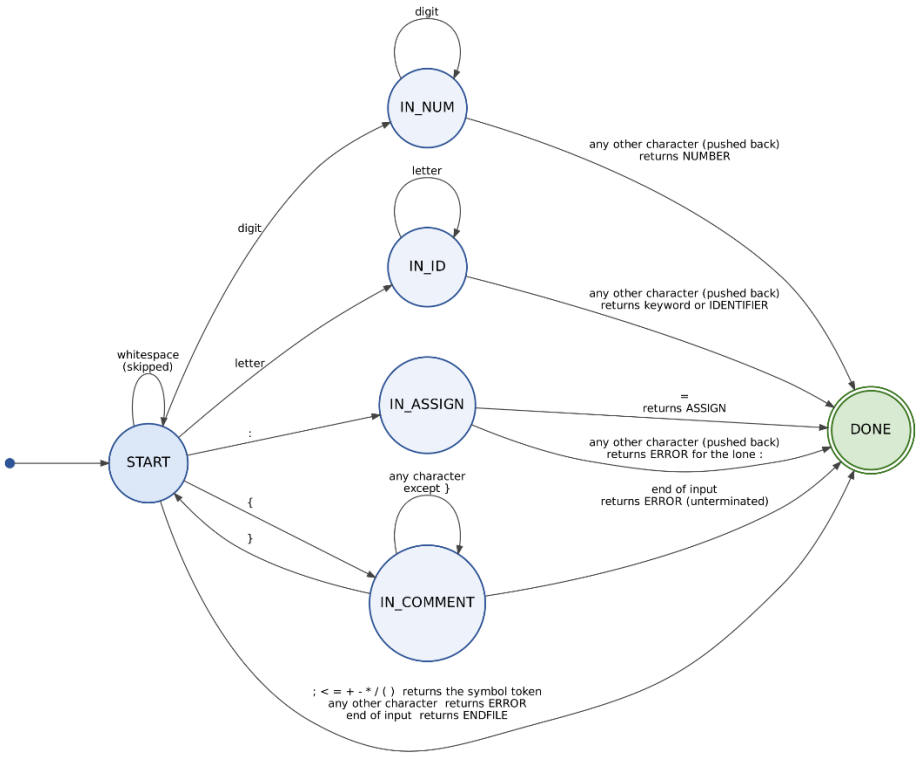
The command line reports each error through a shared formatter as `[Line X, Column Y]: Lexical Error -> Unrecognized symbol 'Z'`, writes the tokens collected so far to the output file, and exits with status 1.

These are lexical errors only. The scanner classifies characters and has no grammar, so a program that is well-formed character by character but nonsense as TINY, such as `end repeat x`, produces no error here. Catching that is the parser's job in a later phase.

**DFA**

The scanner is a deterministic finite automaton with six states. Each call to getToken() starts in START and ends in DONE.

| State | Meaning |
|---|---|
| START | Ready to read the first character of a token. Whitespace is skipped here. |
| IN_NUM | Collecting the digits of a number. |
| IN_ID | Collecting the letters of an identifier or keyword. |
| IN_ASSIGN | Saw : and expects = next. |
| IN_COMMENT | Inside { ... }, skipping everything until }. |
| DONE | A token has been recognized and is returned. |




## Building and running

The program is a command line tool.

**Interactive menu (running with no arguments)**

When the program is started without arguments, for example by double-clicking TinyScanner.exe, it shows a menu instead of closing. The menu repeats until the user chooses Exit.

| Option | What it does |
|---|---|
| 1. Scan the sample files already present | Scans every .tiny file in the samples folder and next to the program. A result file named <name>_tokens.txt is written beside each input file. |
| 2. Scan another file | Asks for an input file and an output file, then scans. If the output name is left blank, it defaults to <input name>_tokens.txt. |
| 3. Show help | Prints the usage of the -i and -o options. |
| 4. Exit | Closes the program with exit status 0. |

For option 1, the samples folder must sit in the same folder as the executable.

**Second Option : Scan another file (**Running with arguments**)**

The scanner can be run directly from the command line by giving it an input file and an output file:

| |
|---|
| python main.py -i samples/sample_factorial.tiny -o output_tokens.txt |
| TinyScanner.exe -i sample_factorial.tiny -o output_tokens.txt |

The program reads the source file, writes the token list to the output file, and prints a summary to the console. Every lexical error is reported with its line and column. The exit status tells a marker or a script what happened without reading the output.

| Exit status | Meaning |
|---|---|
| 0 | The file scanned cleanly, no lexical errors |
| 1 | The scan found at least one lexical error, or no arguments were given |
| 2 | The input file could not be read |

**Build the standalone executable**

```
build_exe.bat
```

The script installs PyInstaller if it is missing, then packages the project into `dist/TinyScanner/TinyScanner.exe`. The build runs in console mode, so the executable keeps the terminal window that carries its output. The result runs on any Windows machine with no Python installation and no third-party tools, which is what the specification requires.

```
TinyScanner.exe -i sample_factorial.tiny -o output_tokens.txt
```

## Example

Scanning `samples/sample_test.tiny`, which exercises every token type in the specification table, yields 32 tokens and no errors.

**Input**

```
{ Test of all tokens from the specification table }
if x < 12 then
  repeat
    read abc;
    xyz := abc + 4 - (289 * x) / 2;
    write xyz;
  until xyz = 0
end;
```

**Command**

```
python main.py -i samples/sample_test.tiny -o output_tokens.txt
```

**Console output**

```
============================================================
  TINY Language Lexical Analyzer (Scanner Project)
  Execution Mode: CLI
============================================================
Reading input file:  'samples/sample_test.tiny'
Destination output: 'output_tokens.txt'

[+] Scan completed: 32 tokens scanned.
[+] Success: No lexical errors detected.
[+] Output written to: 'output_tokens.txt'
```

**Token file**

The opening comment produced no tokens, and the two-character `:=` was recognised as one ASSIGN token rather than a `:` followed by an `=`.

```
if, IF
x, Identifier
<, LESSTHAN
12, number
then, THEN
repeat, REPEAT
read, READ
abc, Identifier
;, SEMICOLON
xyz, Identifier
:=, assign
abc, Identifier
+, PLUS
4, number
-, MINUS
(, OPENBRACKET
289, number
*, MULT
x, Identifier
), CLOSEDBRACKET
/, DIV
2, number
;, SEMICOLON
write, WRITE
xyz, Identifier
;, SEMICOLON
until, UNTIL
xyz, Identifier
=, EQUAL
0, number
end, END
;, SEMICOLON
```